Project Title

MedAssist AI – Smart Healthcare & Diagnostic System

Submitted By: Mohammad Aleem Qurammi

Milestone: Milestone 3

Role:

Full-Stack Development & Machine Learning Integration – Random Forest Diagnostic Model, FastAPI Microservice, Dual-Option Diagnostic UI, and Multi-Cloud Deployment Infrastructure for the MedAssist AI Web Platform.

1. Introduction

MedAssist AI is a production-grade AI healthcare platform delivering rapid, data-driven clinical symptom evaluation, disease risk prediction, document and scan parsing, and conversational AI health assistance. Milestone 3 focused on major architectural enhancements built on top of the AI Assistant memory engine and document generation features delivered in Milestone 2, culminating in a trained machine learning diagnostic pipeline, a dedicated Python microservice, a dual-mode diagnostic interface, an enhanced clinical action suite, and production deployment readiness across multiple cloud platforms.

2. Work Completed

2.1 Machine Learning Model Engineering (ml_backend/)

Processed a dataset of 60,000 clinical records spanning 377 binary symptom features and 658 target disease labels. The model architecture used is a RandomForestClassifier with 200 estimators, a maximum depth of 25, minimum samples split of 4, and the “sqrt” max-features setting, trained and evaluated on an 80/20 train-test split.

- Overall Accuracy: 74.72%
- Weighted Precision: 78.99%
- Weighted Recall: 74.72%
- Weighted F1-Score: 74.11%
- Top-1 Accuracy: 74.72%
- Top-3 Accuracy: 86.72%
- Top-5 Accuracy: 89.83%

Generated artifacts include the trained model weights (best_model.joblib, 2.6 GB), the 658-class label encoder (label_encoder.joblib), and the 377-feature schema (feature_names.json).

2.2 FastAPI ML Microservice (ml_backend/app.py)

Built a high-performance Python FastAPI service exposing /predict and /health endpoints. An intelligent symptom feature matcher converts free-text or selected symptom names into a 377-dimensional binary array using regex, snake_case transformation, and substring matching. The service returns ranked Top-5 clinical differential diagnoses with risk classification (High, Moderate, Low) and urgency guidance (Consult Doctor Urgently, Monitor & Rest, Routine Care).

An auto-download boot engine (start.py) retrieves the 2.6 GB model weights from Google Drive on first server boot using gdown.

2.3 Dual-Option UI System (/dashboard/symptom-checker & /dashboard/analysis)

Delivered two complementary diagnostic interfaces with a mode-switcher navigation bar for seamless toggling between them:

- Option 1 – All-in-One Instant Symptom Checker: a single-view interface with auto-filled patient details, a symptom pills selector, a medical report file uploader, and an instant Random Forest prediction breakdown.
- Option 2 – Step-by-Step Diagnostic Wizard: a guided flow moving through Disclaimer, Profile, Symptom Details, Report Upload, a processing animation, and Results.

2.4 Interactive Clinical Action Suite (PDF, Share, Save)

- PDF Report: client-side generation via jsPDF and jspdf-autotable, producing clinical reports with patient vitals, detected symptoms, the ML diagnosis table, confidence scores, and action plans.
- Share: Web Share API integration with automatic fallback to clipboard copy, including a "Copied!" confirmation badge.
- Save to Dashboard: stores diagnostic results into local medical history (saved_medical_reports) and triggers the in-app NotificationContext engine.

2.5 Multi-Cloud Production Deployment Infrastructure

- Render.com Blueprint (render.yaml): infrastructure-as-code deploying both the Next.js and FastAPI services with environment variable linking.
- Vercel + Hugging Face Spaces: frontend on Vercel serverless deployment; ML microservice on a Hugging Face Spaces Docker container with 16 GB RAM.
- AWS EC2 (Docker Compose): multi-container deployment for dedicated server environments, using separate frontend and ML backend Dockerfiles.

3. Technology Stack

Layer	Technology Used
Frontend Framework	Next.js 16 (App Router, Turbopack), React 19, TypeScript
Styling & Design System	TailwindCSS v4
Backend Language	Python 3.11
ML Microservice Framework	FastAPI
Machine Learning	Scikit-Learn (Random Forest, 200 Trees)
Authentication & Database	Firebase Auth / Firestore
LLM Engine	Groq API (Llama 3.3 70B)
Document Generation	ReportLab, jsPDF
Containerization	Docker

4. Verification & Performance Status

All core ML endpoints and UI components were verified with the following results:

- ML Microservice: Random Forest model trained on 60,000 samples across 658 disease classes, achieving 89.83% Top-5 accuracy.
- Predict Endpoint: returns ranked Top-5 differential diagnoses with risk classification and urgency guidance.
- Build & Compilation: all source code, trained model artifacts, PDF documentation, and deployment configurations verified, committed, and tested cleanly.

5. Summary of Accomplishments

Feature	Milestone 2 State	Milestone 3 Complete State
ML Model	Simulated / Fallback	Trained 200-Tree Random Forest (60,000 samples, 658 classes)
Top-5 Accuracy	N/A	89.83% Top-5 / 86.72% Top-3 clinical accuracy
Backend API	Next.js API Routes	FastAPI Python microservice (Port 8000)
UI Experience	Standard Forms	Dual-option switcher (All-in-One + Step Wizard)
Report Actions	Static Buttons	Workable PDF generation, link share & dashboard save
Deployment	Local Dev	Render blueprint, Vercel + Hugging Face (16GB RAM), AWS EC2

6. Conclusion

During this milestone, MedAssist AI gained a trained 200-tree Random Forest diagnostic model spanning 658 disease classes, a dedicated FastAPI machine learning microservice, and a dual-option diagnostic interface offering both an instant checker and a guided step-by-step wizard. The clinical action suite was strengthened with workable PDF report generation, link sharing, and dashboard record preservation, while multi-cloud deployment blueprints for Render, Vercel with Hugging Face Spaces, and AWS EC2 established the platform's production readiness.